Winstec
Robot API

Table of Contents

1. Introduction

This document describes the robot APIs, including command descriptions, parameter definitions and usage examples, to help developers integrate and use the APIs efficiently.

2. Release Notes

Release 1.1

Release Date Jul 14, 2026
Editor Bob Su

New Features

- Added movement control APIs.
- Added relocation APIs.
- Added map management APIs
- Added point management APIs.
- Added virtual wall management APIs
- Added group management APIs

Release 1.0

Release Date May 25, 2026
Editor Bob Su

New Features

- First edition

3. Connection Settings

The robot provides an HTTP server for communication with external systems, and a WebSocket server for real-time event notifications.

This section describes the connection information required to access the APIs, including the IP addresses and ports of the HTTP server and WebSocket server.

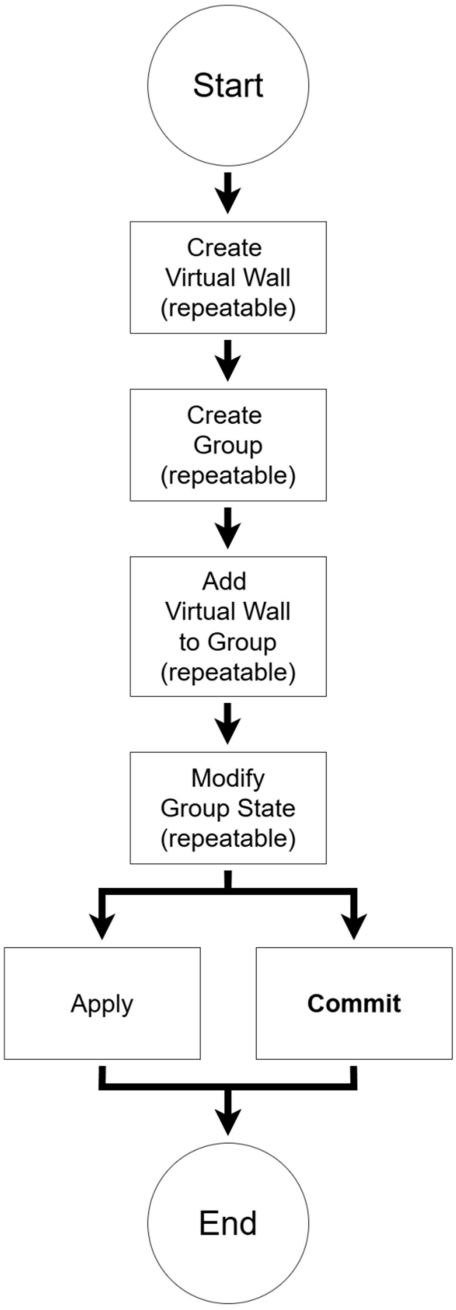
Server Configurations

Service	Protocol	IP Address	Port	Description
HTTP Server	HTTP	192.168.125.88	5000	Provides REST APIs for robot control and status monitoring.
WebSocket Server	WebSocket	192.168.125.88	5001	Provides real-time event and status notifications

4. Group Management

A **Group** is used to manage multiple **Virtual Walls**. By enabling or disabling a **Group**, users can activate or deactivate all **Virtual Walls** within that **Group**.

4.1 Flowchart



4.2 Description

Create Virtual Wall

Multiple Virtual Walls can be created in the system.

Create Group

Multiple Groups can be created in the system. Each Group is used to manage Virtual Walls.

Add Virtual Wall to Group

A virtual wall can be added to one or more Groups.

Modify Group State

The default state of a Group is disabled. A Group can be enabled or disabled. Only enabled Groups are applied to the robot.

Apply

Applies the current Group states to the robot

Commit

Commits all Virtual Walls and Groups, then applies the updated Groups to the robot.

5. REST API Reference

The HTTP server provides REST APIs for robot control and status monitoring.

API List

Endpoint	Method	Description
/v1/robot/info	GET	Retrieves the robot information
/v1/robot/move	POST	Navigates the robot to the specified location
/v1/robot/move/{pointId}	POST	Navigates the robot to the specified point
/v1/robot/manual/move	POST	Controls the robot's movement manually
/v1/robot/stop	POST	Cancel the current navigation mission.
/v1/robot/relocate/location	POST	Sets the robot position to the specified location
/v1/robot/relocate/{pointId}	POST	Sets the robot position to the specified point
/v1/robot/points	POST	Creates a new point
/v1/robot/points?*map=name*	GET	Retrieves all points in map
/v1/robot/points/{pointId}	GET	Retrieves a point
/v1/robot/points/{pointId}	PATCH	Modifies the specified point
/v1/robot/points/{pointId}?*map=name*	DELETE	Deletes the specified point
/v1/robot/virtual-walls	POST	Creates a virtual wall
/v1/robot/virtual-walls?*map=name*	GET	Retrieves all virtual walls in the map
/v1/robot/virtual-walls/{virtualWallId}	PATCH	Modifies the specified virtual wall
/v1/robot/virtual-walls/{virtalWallId}?*map=name*	DELETE	Deletes the specified virtual wall

Endpoint	Method	Description
/v1/robot/groups	POST	Creates a new group
/v1/robot/groups?*map=name*	GET	Retrieves all groups in the map
/v1/robot/groups/{groupId}	GET	Retrieves the group details
/v1/robot/groups/{groupId}	PATCH	Modifies the specified group
/v1/robot/groups/{groupId}	DELETE	Deletes the specified group
/v1/robot/groups/{groupId}/virtual-walls	POST	Add a virtual wall to the specified group
/v1/robot/groups/{groupId}/virtual-walls	GET	Retrieves all virtual walls in the specified group
/v1/robot/groups/{groupId}/virtual-walls/{virtualWallId}	DELETE	Deletes the specified virtual wall in the specified group
/v1/robot/groups/actions/apply	POST	Applies all groups to the robot
/v1/robot/edits/commit	POST	Commits all changes to points, virtual walls, and groups, then applies the updated groups to the robot
/v1/robot/edits/discard	POST	Discards all changes to points, virtual walls, and groups.
/v1/robot/shutdown	POST	Shuts down the robot.

Robot Information

Retrieves robot information.

1. HTTP Request

GET /v1/robot/info

2. Response

2.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
op_mode	string	**Yes**	Robot operating mode (see Section 7.1: Operating Mode)
status	string	**Yes**	Robot status (see Section 7.2: Robot Status)
battery	integer	**Yes**	Battery level
location	object	**Yes**	Robot location object (see Section 7.4: Location Object)

2.2 Error Response

See Section 6: Error Response

3. Example

Request:

GET /v1/robot/info

Response:

HTTP/1.1 **200 OK**


```
{
   "op_mode": "navigate",
   "status": "idle",
   "battery": 90,
   "location": {
      "x": 1067,
      "y": -182,
      "orientation": 12.384
   }
}
```

● Move to Location

Moves the robot to the specified target location.

1. HTTP Request

POST /v1/robot/move

2. Request Body

Name	Type	Required	Description
type	string	**Yes**	Location type (see Section 7.3: Point Type)
location	object	**Yes**	Target location object (see Section 7.4: Location Object)

3. Response Body

3.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
type	string	**Yes**	Location type (see Section 7.3: Point Type)
location	object	**Yes**	Target location object (see Section 7.4: Location Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

4.1. Move to target location

Request:

```
POST /v1/robot/move
{
   "type": "point",
   "location": {
      "x": 1277,
      "y": -134,
      "orientation": 19.182
   }
}
```

Response:

```
HTTP/1.1 200 OK
{
   "type": "point",
   "location": {
      "x": 1277,
      "y": -134,
      "orientation": 19.182
   }
}
```

● Move to Point

Moves the robot to the existing point.

1. HTTP Request

POST /v1/robot/move/{pointId}

2. Path Parameters

Name	Type	Required	Description
pointId	string	**Yes**	Point ID

3. Response

3.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
id	string	**Yes**	Point ID
map	string	**Yes**	Map name
name	string	**Yes**	Point name
type	string	**Yes**	Point type (see Section 7.3: Point Type)
location	object	**Yes**	Point location object (see Section 7.4: Location Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

POST /v1/robot/move/pt__5Yk0zPgdZw9udgLz_9sBg

Response:

HTTP/1.1 **200 OK**


```
{
   "id": "pt__5Yk0zPgdZw9udgLz_9sBg",
   "map": "20260605_bob",
   "name": "p1",
   "type": "point",
   "location": {
      "x": 1144,
      "y": -39,
      "orientation": 10
   }
}
```

Move Manually

Controls the robot movement manually.

1. HTTP Request

POST /v1/robot/manual/move

2. Request Body

Name	Type	Required	Description
direction	string	**Yes**	Movement direction (see Movement Direction)

Movement Direction

Value	Description
stop	Stop movement
forward	Move forward
backward	Move backward
right	Rotate clockwise
left	Rotate counterclockwise

3. Response Body

3.1 Success Response

Status Code: **200 OK**

Response Body: None

3.2 Error Response

See Section 6 Error Responses

3. Example

Request:

```
POST /v1/robot/manual/move
{
   "direction": "forward"
}
```

Response:

HTTP/1.1 **200 OK**

● Stop

Soft-stops the robot and cancels the current navigation mission.

1. HTTP Request

POST /v1/robot/stop

2. Response Body

Status Code: **200 OK**

Response Body: None

3. Example

Request:

POST /v1/robot/stop

Response:

HTTP/1.1 **200 OK**

● Relocation by Location

Sets the robot pose to the specified location.

1. HTTP Request

POST /v1/robot/relocate/location

2. Request Body

Name	Type	Required	Description
location	object	**Yes**	Location object (see Section 7.4: Location Object)

3. Response Body

3.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
location	object	**Yes**	Location object (see Section 7.4: Location Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

```
POST /v1/robot/relocate/location
{
   "location": {
      "x": -4,
      "y": 0,
      "orientation": 356
   }
}
```

Response:

```
HTTP/1.1 200 OK
{
   "location": {
      "x": -4,
      "y": 0,
      "orientation": 356
   }
}
```

● Relocation by Point

Sets the robot pose to the specified point

1. HTTP Request

POST /v1/robot/relocate/{pointId}

2. Path Parameters

Name	Type	Required	Description
pointId	string	**Yes**	Point ID.

3. Response Body

3.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
id	string	**Yes**	Point ID
map	string	**Yes**	Map name
name	string	**Yes**	Point name
type	string	**Yes**	Point type (see Section 7.3: Point Type)
location	object	**Yes**	Point location object (see Section 7.4: Location Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

POST /v1/robot/relocate/pt_u7-6MhQG_M2UXgTy62JibQ

Response:

HTTP/1.1 **200 OK**


```
{
   "id": "pt_u7-6MhQG_M2UXgTy62JibQ",
   "map": "test",
   "name": "c1",
   "type": "charge",
   "location": {
      "x": -4,
      "y": 0,
      "orientation": 356
   }
}
```

Create Point

Creates a new point on the map.

1. HTTP Request

POST /v1/robot/points

2. Request Body

Name	Type	Required	Description
map	string	**No**	Map name. If omitted, the currently loaded map is used.
name	string	**Yes**	Point name
type	string	**Yes**	Point type (see Section 7.3: Point Type)
location	object	**No**	Point location object (see Section 7.4: Location Object). If omitted, the robot's current position is used.

3. Response Body

3.1 Success Response

Status Code: **201 CREATED**

Response Body:

Name	Type	Required	Description
id	string	**Yes**	Point ID
map	string	**Yes**	Map name
name	string	**Yes**	Point name
type	string	**Yes**	Point type (see Section 7.3: Point Type)
location	object	**Yes**	Point location object (see Section 7.4: Location Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

```
POST /v1/robot/points
{
   "name": "p3",
   "type": "point"
}
```

Response:

```
HTTP/1.1 201 CREATED
{
   "id": "pt_d6lr2_O6SPemXHms3uzXMw",
   "map": "test",
   "name": "p3",
   "type": "point",
   "location": {
      "x": 1403,
      "y": -348,
      "orientation": 24.262
   }
}
```

List Points

Retrieves all points from the specified map.

1. HTTP Request

GET /v1/robot/points*?map=name*

2. Query Parameters

Name	Type	Required	Description
map	string	**No**	Map name. If omitted, the currently loaded map is used

3. Response Body

3.1 Success Response

Status Code: **200 OK**
Response Body:

Name	Type	Required	Description
points	array	**Yes**	Array of point objects (see Point Object)

Point Object

Name	Type	Required	Description
id	string	Yes	Point ID
map	string	Yes	Map name
name	string	Yes	Point name
type	string	Yes	Point type (see Section 7.3: Point Type)
location	object	Yes	Point location (see Section 7.4: Location Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

GET /v1/robot/points

Response:

HTTP/1.1 **200 OK**


```
{
   "points": [
      {
         "id": "pt__5Yk0zPgdZw9udgLz_9sBg",
         "map": "test",
         "name": "p1",
         "type": "point",
         "location": {
            "x": 1144,
            "y": -39,
            "orientation": 10
         }
      },
      {
         "id": "pt_u7-6MhQG_M2UXgTy62JibQ",
         "map": "test",
         "name": "c1",
         "type": "charge",
         "location": {
            "x": -4,
            "y": 0,
            "orientation": 356
         }
      }
   ]
}
```

● Get Point

Retrieves the specified point.

1. HTTP Request

GET /v1/robot/points/{pointId}

2. Path Parameters

Name	Type	Required	Description
pointId	string	**Yes**	Point ID

3. Response Body

3.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
id	string	**Yes**	Point ID
map	string	**Yes**	Map name
name	string	**Yes**	Point name
type	string	**Yes**	Point type (see Section 7.3: Point Type)
location	object	**Yes**	Point location object (see Section 7.4: Location Object).

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

GET /v1/robot/points/pt__5Yk0zPgdZw9udgLz_9sBg

Response:

HTTP/1.1 **200 OK**


```
{
   "id": "pt__5Yk0zPgdZw9udgLz_9sBg",
   "map": "test",
   "name": "p1",
   "type": "point",
   "location": {
      "x": 1144,
      "y": -39,
      "orientation": 10
   }
}
```

Modify Point

Modifies the specified point.

1. HTTP Request

PATCH /v1/robot/points/{pointId}

2. Path Parameters

Name	Type	Required	Description
pointId	string	**Yes**	Point ID

3. Request Body

Name	Type	Required	Description
map	string	**No**	Map name
name	string	**No**	Point name
type	string	**No**	Point type (see Section 7.3: Point Type)
location	object	**No**	Point location object (see Section 7.4: Location Object)

4. Response Body

4.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
id	string	**Yes**	Point ID
map	string	**Yes**	Map name
name	string	**Yes**	Point name
type	string	**Yes**	Point type (see Section 7.3: Point Type)
location	object	**Yes**	Point location object (see Section 7.4: Location Object)

4.2 Error Response

See Section 6 Error Responses

5. Example

Request:

```
PATCH /v1/robot/points/pt__5Yk0zPgdZw9udgLz_9sBg
{
   "name": "p1.5"
}
```

Response:

```
HTTP/1.1 200 OK
{
   "id": "pt__5Yk0zPgdZw9udgLz_9sBg",
   "map": "test",
   "name": "p1.5",
   "type": "point",
   "location": {
      "x": 1144,
      "y": -39,
      "orientation": 10
   }
}
```

Delete Points

Deletes one or more points from the map

1. HTTP Request

DELETE /v1/robot/points/{pointId}*?map=name*

2. Path Parameters

Name	Type	Required	Description
pointId	string	**No**	Point ID to delete

3. Query Parameters

Name	Type	Required	Description
map	String	**No**	Map name. Delete all points in the specified map if pointId is omitted.

4. Response Body

4.1 Success Response

Status Code: **204 NO CONTENT**

Response Body: None

4.2 Error Response

See Section 6 Error Responses

5. Example

Request:

DELETE /v1/robot/points/pt__5Yk0zPgdZw9udgLz_9sBg

Response:

HTTP/1.1 **204 NO CONTENT**

Create Virtual Wall

Creates a new virtual wall on the map.

1. HTTP Request

POST /v1/robot/virtual-walls

2. Request Body

Name	Type	Required	Description
map	string	**No**	Map name. If omitted, the currently loaded map is used.
name	string	**Yes**	Virtual wall name
start_position	object	**Yes**	Virtual wall start position (see Section 7.5: Position Object)
end_position	object	**Yes**	Virtual wall end position (see Section 7.5: Position Object)

3. Response Body

3.1 Success Response

Status Code: **201 CREATED**

Response Body:

Name	Type	Required	Description
id	string	**Yes**	Virtual wall ID
map	string	**Yes**	Map name
name	string	**Yes**	Virtual wall name
start_position	object	**Yes**	Virtual wall start position (see Section 7.5: Position Object)
end_position	object	**Yes**	Virtual wall end position (see Section 7.5: Position Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

```
POST /v1/robot/virtual-walls
{
   "name": "w_test",
   "start_position": {
      "x": 624,
      "y": 159
   },
   "end_position": {
      "x": 524,
      "y": -241
   }
}
```

Response:

```
HTTP/1.1 201 CREATED
{
   "id": "vw_6k2BX40aQqW_wyjrK93RNQ",
   "map": "test",
   "name": "w_test",
   "start_position": {
      "x": 624,
      "y": 159
   },
   "end_position": {
      "x": 524,
      "y": -241
   }
}
```

List Virtual Walls

Retrieves all virtual walls from the specified map.

1. HTTP Request

GET /v1/robot/virtual-walls*?map=name*

2. Query Parameters

Name	Type	Required	Description
map	string	**No**	Map name. If omitted, the currently loaded map is used.

3. Response Body

3.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
virtual_walls	array	**Yes**	Array of virtual wall objects (see Virtual Wall Object)

Virtual Wall Object

Name	Type	Required	Description
id	string	**Yes**	Virtual wall ID
map	string	**Yes**	Map name
name	string	**Yes**	Virtual wall name
start_position	object	**Yes**	Virtual wall start position (see Section 7.5: Position Object)
end_position	object	**Yes**	Virtual wall end position (see Section 7.5: Position Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

GET /v1/robot/virtual-walls

Response:

Status Code: 200 OK

```
{
   "virtual_walls": [
      {
         "id": "vw_6k2BX40aQqW_wyjrK93RNQ",
         "map": "20260605_bob",
         "name": "w_test",
         "start_position": {
            "x": 624,
            "y": 159
         },
         "end_position": {
            "x": 524,
            "y": -241
         }
      },
      ……
   ]
}
```

Modify Virtual Wall

Modifies the specified virtual wall.

1. HTTP Request

PATCH /v1/robot/virtual-walls/{virtualWallId}

2. Path Parameters

Name	Type	Required	Description
virtualWallId	string	**Yes**	Virtual wall ID

3. Request Body

Name	Type	Required	Description
map	string	**No**	Map name. If omitted, the currently loaded map is used.
name	string	**No**	Virtual wall name
start_position	object	**No**	Start position (see Section 7.5: Position Object)
end_position	object	**No**	End position (see Section 7.5: Position Object)

4. Response Body

4.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
id	string	**Yes**	Virtual wall ID
map	string	**No**	Map name
name	string	**No**	Virtual wall name
start_position	object	**No**	Virtual wall start position (see Section 7.5: Position Object)
end_position	object	**No**	Virtual wall end position (see Section 7.5: Position Object)

4.2 Error Response

See Section 6 Error Responses

5. Example

Request:

```
PATCH /v1/robot/virtual-walls/vw_6k2BX40aQqW_wyjrK93RNQ
{
   "start_position": {
      "x": 524,
      "y": 59
   },
   "end_position": {
      "x": 424,
      "y": -141
   }
}
```

Response:

```
HTTP/1.1 200 OK
{
   "id": "vw_6k2BX40aQqW_wyjrK93RNQ ",
   "map": "test",
   "name": "w_test",
   "start_position": {
      "x": 524,
      "y": 59
   },
   "end_position": {
      "x": 424,
      "y": -141
   }
}
```

Delete Virtual Wall

Deletes one or more virtual walls from the map

1. HTTP Request

DELETE /v1/robot/virtual-walls/{virtualWallId}*?map=name*

2. Path Parameters

Name	Type	Required	Description
virtualWallId	string	**No**	Virtual wall ID to delete

3. Query Parameters

Name	Type	Required	Description
map	String	**No**	Map name. Delete all virtual walls in the specified map if virtualWallId is omitted.

4. Response Body

4.1 Success Response

Status Code: **204 NO CONTENT**

Response Body: None

4.2 Error Response

See Section 6 Error Responses

5. Example

Request:

DELETE /v1/robot/virtual-walls/vw_6k2BX40aQqW_wyjrK93RNQ

Response:

HTTP/1.1 **204 NO CONTENT**

Create Group

Creates a new group.

1. HTTP Request

POST /v1/robot/groups

2. Request Body

Name	Type	Required	Description
map	string	**No**	Map name. If omitted, the currently loaded map is used.
name	string	**Yes**	Group name

3. Response Body

3.1 Success Response

Status Code: **201 CREATED**

Response Body:

Name	Type	Required	Description
id	string	**Yes**	Group ID
map	string	**Yes**	Map name
name	string	**Yes**	Group name

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

```
POST /v1/robot/groups
{
   "name": "group_test"
}
```

Response:

```
HTTP/1.1 201 CREATED
{
   "id": "gp_KwWWwyisSr68_hd669bAGQ",
   "map": "test",
   "name": "group_test"
}
```

List Groups

Retrieves all groups from the specified map.

1. HTTP Request

GET /v1/robot/groups*?map=name*

2. Query Parameters

Name	Type	Required	Description
map	string	**No**	Map name. If omitted, the currently loaded map is used.

3. Response Body

3.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
groups	array	**Yes**	Array of Group objects (see Group Object)

Group Object

Name	Type	Required	Description
id	string	**Yes**	Group ID
map	string	**Yes**	Map name
name	string	**Yes**	Group name
is_enable	boolean	**Yes**	Group enabled status

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

GET /v1/robot/groups

Response:

HTTP/1.1 **200 OK**

```
{
   "groups": [
      {
         "id": "gp_KwWWwyisSr68_hd669bAGQ",
         "map": "20260605_bob",
         "name": "group_test",
         "is_enable": false
      },
      ……
   ]
}
```

Get Group Details

Retrieves the specified group details

1. HTTP Request

GET /v1/robot/groups/{groupId}

2. Path Parameters

Name	Type	Required	Description
groupId	string	**Yes**	Group ID

3. Response Body

3.1 Success Response

Status Code: **200 OK**
Response Body:

Name	Type	Required	Description
id	string	**Yes**	Group ID
map	string	**Yes**	Map name
name	string	**Yes**	Group name
is_enable	boolean	**Yes**	Indicates whether the group is enabled
virtual_walls	array	**Yes**	Array of virtual wall objects (see Virtual Wall Object)

Virtual Wall Object

Name	Type	Required	Description
id	string	**Yes**	Virtual wall ID
map	string	**Yes**	Map name
name	string	**Yes**	Virtual wall name
start_position	object	**Yes**	Virtual wall start position (see Section 7.5: Position Object)
end_position	object	**Yes**	Virtual wall end position (see Section 7.5: Position Object)

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

GET /v1/robot/groups

Response:

HTTP/1.1 **200 OK**


```
{
   "id": "gp_KwWWwyisSr68_hd669bAGQ",
   "map": "test",
   "name": "group_test",
   "is_enable": false,
   "virtual_walls": [
      {
         "id": "vw_P_XcvqJkcp3GcwSHskpgaA",
         "map": "test",
         "name": "w1",
         "start_position": {
            "x": 724,
            "y": 259
         },
         "end_position": {
            "x": 624,
            "y": -341
         }
      }
   ]
}
```

● Modify Group

Modifies the specified group. Changes will not take effect until the **Apply Groups** API is called.

1. HTTP Request

PATCH /v1/robot/groups/{groupId}

2. Path Parameters

Name	Type	Required	Description
groupId	string	**Yes**	Group ID

3. Request Body

Name	Type	Required	Description
name	string	**No**	Group name
is_enable	boolean	**No**	Group enabled status.

4. Response Body

4.1 Success Response

Status Code: **200 OK**
Response Body:

Name	Type	Required	Description
id	string	**Yes**	Deleted virtual wall ID
map	string	**Yes**	Map name
name	string	**Yes**	Group name
is_enable	boolean	**Yes**	Group enabled status

4.2 Error Response

See Section 6 Error Responses

5. Example

Request:

```
PATCH /v1/robot/groups/vw_6k2BX40aQqW_wyjrK93RNQ
{
   "is_enable": true
}
```

Response:

```
HTTP/1.1 200 OK
{
   "id": " vw_6k2BX40aQqW_wyjrK93RNQ ",
   "map": "test",
   "name": "group_test",
   "is_enable": true
}
```

Delete Group

Deletes the specified group.

1. HTTP Request

DELETE /v1/robot/groups/{groupId}

2. Path Parameters

Name	Type	Required	Description
groupId	string	**Yes**	Group ID

3. Response Body

3.1 Success Response

Status Code: 204 NO CONTENT

Response Body: None

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

DELETE /v1/robot/groups/gp_KwWWwyisSr68_hd669bAGQ

Response:

HTTP/1.1 **204 NO CONTENT**

Add Virtual Wall to Group

Adds a virtual wall to the specified group.

1. HTTP Request

POST /v1/robot/groups/{groupId}/virtual-walls

2. Path Parameters

Name	Type	Required	Description
groupId	string	**Yes**	Group ID

3. Request Body

Name	Type	Required	Description
id	string	**Yes**	Virtual wall ID

4. Response Body

4.1 Success Response

Status Code: **201 CREATED**

Response Body:

Name	Type	Required	Description
group_id	string	**Yes**	Group ID
virtual_wall_id	string	**Yes**	Virtual wall ID

4.2 Error Response

See Section 6 Error Responses

5. Example

Request:

```
POST /v1/robot/groups/gp_4nUk218yPD0E5Mk9bLZ9_g/virtual-walls
{
   "id": "vw_69aHmgpLGSIqM9QBke8OAQ"
}
```

Response:

```
HTTP/1.1 201 CREATED
{
   "group_id": "gp_4nUk218yPD0E5Mk9bLZ9_g",
   "virtual_wall_id": "vw_69aHmgpLGSIqM9QBke8OAQ"
}
```

List Virtual Walls in Group

Retrieves all virtual walls in the specified group.

1. HTTP Request

GET /v1/robot/groups/{groupId}/virtual-walls

2. Path Parameters

Name	Type	Required	Description
groupId	string	**Yes**	Group ID

3. Response Body

3.1 Success Response

Status Code: **200 OK**

Response Body:

Name	Type	Required	Description
virtual_walls	array	**Yes**	Array of Virtual Wall objects (see Virtual Wall Object)

Virtual Wall Object:

Name	Type	Required	Description
id	string	**Yes**	Virtual wall ID

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

GET /v1/robot/groups/gp_4nUk218yPD0E5Mk9bLZ9_g/virtual-walls

Response:

HTTP/1.1 **200 OK**


```
{
   "virtual_walls": [
      {
         "id": "vw_69aHmgpLGSIqM9QBke8OAQ"
      },
      {
         "id": "vw_P_XcvqJkcp3GcwSHskpgaA"
      }
   ]
}
```

Delete Virtual Wall in Group

Deletes a virtual wall from the specified group.

1. HTTP Request

DELETE /v1/robot/groups/{groupId}/virtual-walls/{virtualWallId}

2. Path Parameters

Name	Type	Required	Description
groupId	string	**Yes**	Group ID
virtualWallId	string	**Yes**	Virtual wall ID

3. Response Body

3.1 Success Response

Status Code: **204 NO CONTENT**

Response Body: None

3.2 Error Response

See Section 6 Error Responses

4. Example

Request:

DELETE
/v1/robot/groups/gp_4nUk218yPD0E5Mk9bLZ9_g/virtual-walls/vw_69aHmgpLGSIqM9QBke8OAQ

Response:

HTTP/1.1 204 NO CONTENT

Apply Groups

Applies all group modifications.

1. HTTP Request

POST /v1/robot/groups/actions/apply

2 Response Body

2.1 Success Response

Status Code: 200 OK

Response Body: None

2.2 Error Response

See Section 6 Error Responses

3. Example

Request:

POST /v1/robot/groups/actions/apply

Response:

HTTP/1.1 **200 OK**

Commit

Commits all changes to points, virtual walls, and groups, then applies the updated groups to the robot.

1. HTTP Request

POST /v1/robot/edits/commit

2. Response Body

2.1 Success Response

Status Code: 200 OK

Response Body: None

2.2 Error Response

See Section 6 Error Responses

3. Example

Request:

POST /v1/robot/edits/commit

Response:

HTTP/1.1 **200 OK**

Discard

Discards all changes to points, virtual walls, and groups.

1. HTTP Request

POST /v1/robot/edits/discard

2. Response Body

2.1 Success Response

Status Code: 200 OK

Response Body: None

2.2 Error Response

See Section 6 Error Responses

3. Example

Request:

POST /v1/robot/edits/discard

Response:

HTTP/1.1 **200 OK**

Shutdown

Shuts down the robot.

1. HTTP Request

POST /v1/robot/shutdown

2. Response Body

2.1 Success Response

Status Code: **200 OK**

Response Body: None

2.2 Error Response

See Section 6 Error Responses

3. Example

Request:

POST /v1/robot/shutdown

Response:

HTTP/1.1 **200 OK**

5. Event Reference

The WebSocket server provides real-time event and status notifications.

Event List

Event	Trigger	Description
robot_info	1 second	Reports periodic robot information
go_point	Event	Reports the result of navigation to the point
go_charging	Event	Reports the result of navigation to the charging station
switch_mode	Event	Reports the result of mode switching
relocate	Event	Reports the result of relocation
power	Event	Reports the result of power

Robot Information

The event periodically reports robot information.

1. Message Body

Name	Type	Required	Description
event	string	**Yes**	Fixed value: robot_info
op_mode	string	**Yes**	Robot operating mode (see Section 7.1: Operating Mode)
status	string	**Yes**	Robot status (see Section 7.2: Robot Status)
battery	integer	**Yes**	Battery level
location	object	**Yes**	Robot location object (see Section 7.4: Location Object)

2. Example

```
{
   "event": "robot_info",
   "op_mode": "navigate",
   "status": "idle",
   "battery": 85,
   "location": {
      "x": 3,
      "y": -4,
      "orientation": 1.022
   }
}
```

Go Point

The event reports the result of navigation to the target point.

1. Message Body

Name	Type	Required	Description
event	string	**Yes**	Fixed value: go_point
code	string	**Yes**	Event code (see Section 7.6: Event Code)

2. Example

```
{
   "event": "go_point",
   "code": "complete"
}
```

Go Charging

The event reports the result of navigation to the charging station.

1. Message Body

Name	Type	Required	Description
event	string	**Yes**	Fixed value: go_charging
code	string	**Yes**	Event code (see Section 7.6: Event Code)

2. Example

```
{
   "event": "go_charging",
   "code": "complete"
}
```

Switch Mode

The event reports the result of mode switching.

1. Message Body

Name	Type	Required	Description
event	string	**Yes**	Fixed value: switch_mode
code	string	**Yes**	Event code (see Section 7.6: Event Code)

2. Example

```
{
   "event": "switch_mode",
   "code": "complete"
}
```

Relocation

The event reports the result of relocation

1. Message Body

Name	Type	Required	Description
event	string	**Yes**	Fixed value: relocate
code	string	**Yes**	Event code (see Section 7.6: Event Code)

2. Example

```
{
   "event": "relocate ",
   "code": "complete"
}
```

Power

The event reports the result of power.

1. Message Body

Name	Type	Required	Description
event	string	**Yes**	Fixed value: power
code	string	**Yes**	Event code (see Section 7.6: Event Code)

2. Example

```
{
   "event": "power",
   "code": "SHUTDOWN"
}
```

6. Error Responses

6.1 Response Body

Name	Type	Required	Description
event	object	**Yes**	Error object (see Error Object)

Error Object

Name	Type	Required	Description
code	string	**Yes**	Error code (see 6.2 Error codes)

6.2 Error Codes

400 Bad Request

Error Code	Description
INVALID_INPUT	Invalid input
GET_LOCATION_FAILED	Failed to retrieve robot location
MAP_MISMATCH	Map mismatch

404 Not Found

Error Code	Description
POINT_NOT_FOUND	Point not found
MAP_NOT_FOUND	Map not found
GROUP_NOT_FOUND	Group not found
MISSING_POINT_NAME	Point name missing
MISSING_VIRTUAL_WALL_NAME	Virtual wall name missing
MISSING_GROUP_NAME	Group name missing
NOT_FOUND_IN_GROUP	Item not found in group

409 Conflict

Error Code	Description
ROBOT_BUSY	Robot is executing another task
NOT_IN_NAVIGATION_MODE	Robot is not in navigation mode
POINT_NOT_IN_MAP	Point does not belong to this map

7. Common Data Models

7.1 Operating Mode

Value	Description
explore	Mapping mode
navigate	Navigation mode

7.2 Robot Status

Value	Description
init	System initializing
idle	Idle state
relocating	Relocation in progress
moving	Navigation in progress
go_charging	Navigation to charging station in progress
switching_mode	Operating mode switching

7.3 Point Type

Value	Description
point	Move to a point
charge	Move to a charging station

7.4 Location Object

Name	Type	Required	Description
x	integer	**Yes**	X-axis coordinate
y	integer	**Yes**	Y-axis coordinate
orientation	float	**Yes**	Heading angle (degrees)

7.5 Position Object

Name	Type	Required	Description
x	integer	**Yes**	X-axis coordinate
y	integer	**Yes**	Y-axis coordinate

7.6 Event Codes

Value	Description
COMPLETE	Execution completed
STUCK	Robot stuck
ABORT	Execution aborted
CHG_STA_NOT_FOUND	Charging station not found
SHUTDOWN	Robot shutdown